Analysis of BRKGA for the Euclidean Traveling Salesman Problem

Iago Zagnoli Albergaria[1]

[1]Computer Science Department – Federal University of Minas Gerais (UFMG)

iagozag@ufmg.br

Abstract. *The Euclidean Traveling Salesman Problem (ETSP) seeks the shortest tour visiting a set of points in two-dimensional Euclidean space. This paper examines the BRKGA algorithm, focusing on their complexity and performance when solving the problem. Furthermore, the experimental comparisons highlight trade-offs between solution accuracy and computational efficiency / memory consumption.*

1. Introduction

The Euclidean Traveling Salesman Problem (ETSP) is a classical optimization problem that involves finding the shortest possible path that visits a set of points in a two-dimensional Euclidean space. It is an instance of the Traveling Salesman Problem that has been widely studied due to its computational complexity and extensive applications in logistics, transportation, and circuit design. In the Euclidean version, the distance between two vertices is computed using the Euclidean distance in a two-dimensional plane.

This paper focuses on the analysis of a metaheuristic called BRKGA to solve the ETSP . Genetic algorithms provide faster results, though they may not always produce optimal solutions. The main goal of this study is to evaluate and compare this algorithm, analyzing their computational complexity, memory usage, and solution accuracy.

The remainder of this article is organized as follows. Section 2 details the implementation of the algorithm and the methods selected for each one. Section 3 presents the results of the algorithm in various test cases. Finally, Section 4 provides the conclusions drawn from the analysis.

2. BRKGA Algorithm

The Biased Random-Key Genetic Algorithm (BRKGA) is a metaheuristic designed to find high-quality approximate solutions for combinatorial optimization problems [Gonçalves and Resende 2010], such as the ETSP. This algorithm is inspired by the process of natural evolution, maintaining a population of encoded solutions that evolve over multiple generations through selection, crossover, and mutation.

In the BRKGA, each individual (or chromosome) is represented as a vector of random keys, that is, real numbers uniformly distributed in the interval $[0, 1]$. For the ETSP, each position in this vector corresponds to a city, and its key value determines the order in which that city appears on the tour. To transform this encoded representation into a feasible TSP solution, a decoding step is performed: the cities are sorted according to their key values, and the resulting sequence defines the tour that visits all cities exactly once.

Once the population is initialized with random keys, the algorithm proceeds iteratively through generations. In each generation, all individuals are evaluated by computing the total cost (or distance) of their decoded tours. The best individuals form the elite set, while the remaining ones compose the non-elite set. New individuals are created by performing a biased crossover between an elite parent and a non-elite parent: for each position in the chromosome, the corresponding key is inherited from the elite parent with a given probability (the bias) or from the non-elite parent otherwise. This mechanism ensures that better solutions have a higher influence on the next generation while still preserving population diversity.

To avoid premature convergence and maintain exploration of the search space, a fraction of the population in each generation is replaced by mutants, new randomly generated individuals. After crossover and mutation, the new generation replaces the old one, and the process continues until a stopping criterion is reached (in this implementation, a fixed number of generations).

By combining global exploration through random-key representation and biased genetic operations with selective pressure toward elite solutions, BRKGA is capable of efficiently approximating the optimal TSP tour. Although it does not guarantee an optimal solution, it often produces near-optimal routes within reasonable computational time, particularly for large problem instances where exact algorithms become impractical.

Algorithm 1 Biased Random-Key Genetic Algorithm (BRKGA) for ETSP

Require: Generations G, population size P, elite fraction ρ_e, mutant fraction ρ_m, bias β
Ensure: Best tour found

```
Initialize population with P random individuals (keys ∈ [0, 1])
for each individual do
    Decode (sort cities by keys) and evaluate fitness (tour length)
end for
for g = 1 to G do
    Sort population by fitness; select elites and non-elites
    Generate offspring by biased crossover between elites and non-elites
    Create mutants with random keys
    Form new population: elites + offspring + mutants
end for
return Best individual (lowest tour cost)
```

3. Results

The optimal solutions for each test were from the TSPLIB library [Reinelt 1991], available at http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/STSP.html.

As discussed in the previous section, the BRKGA algorithm requires several parameters to be defined before execution. In this section, we present three experiments with different parameter configurations and compare their results in terms of solution quality (how close they are to the optimal tour) and execution time as the number of vertices increases. The parameters considered are the number of generations, population size, elite fraction, mutant fraction, and bias.

3.1. First test

- Number of Generations: 10000
- Population size: 100
- Elite Fraction: 20%
- Mutant Fraction: 10%
- Bias: 80%

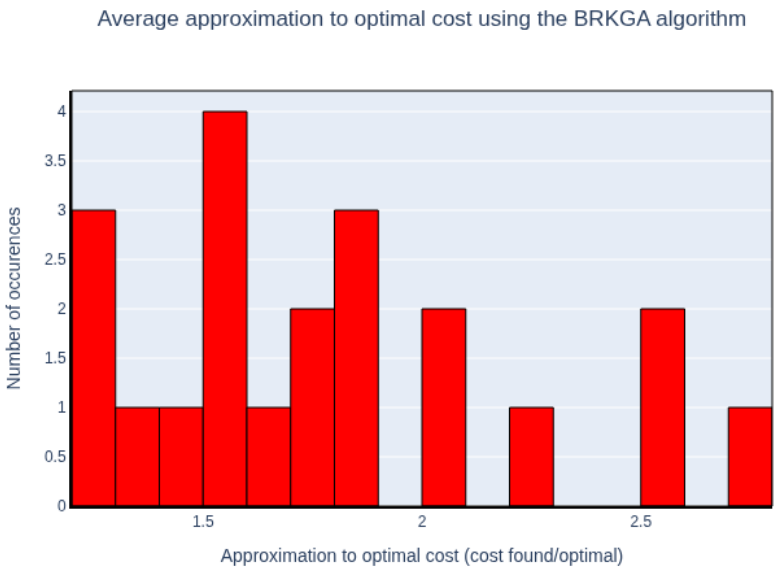

Figure 1. Histogram of approximation rates in first test

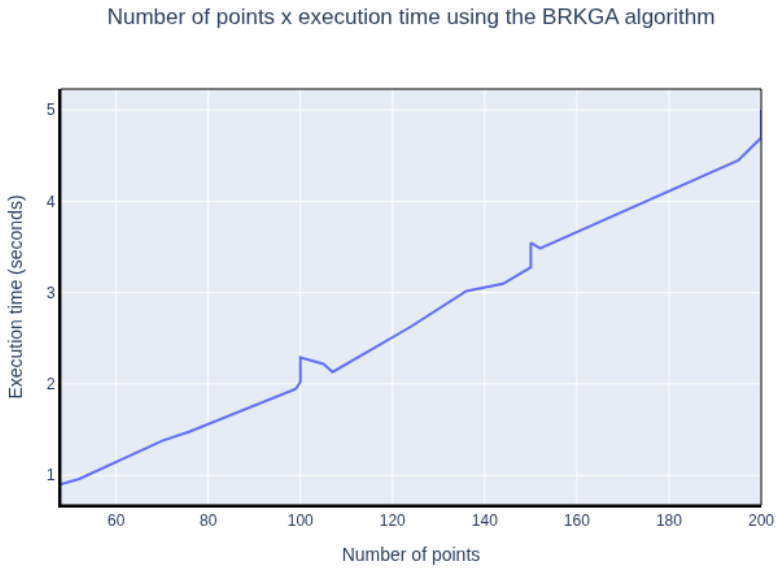

Figure 2. Input Size X Execution time in first test

3.2. Second test

- Number of Generations: 50000
- Population size: 200
- Elite Fraction: 40%
- Mutant Fraction: 30%
- Bias: 60%

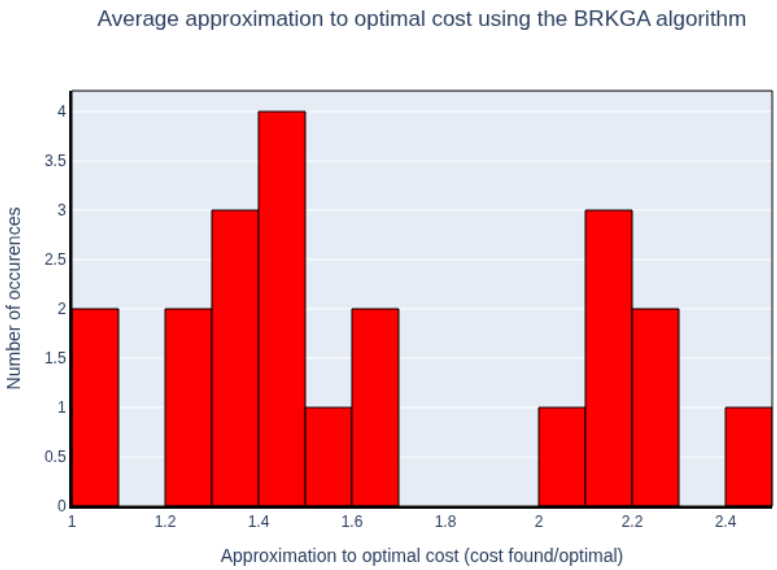

Figure 3. Histogram of approximation rates in second test

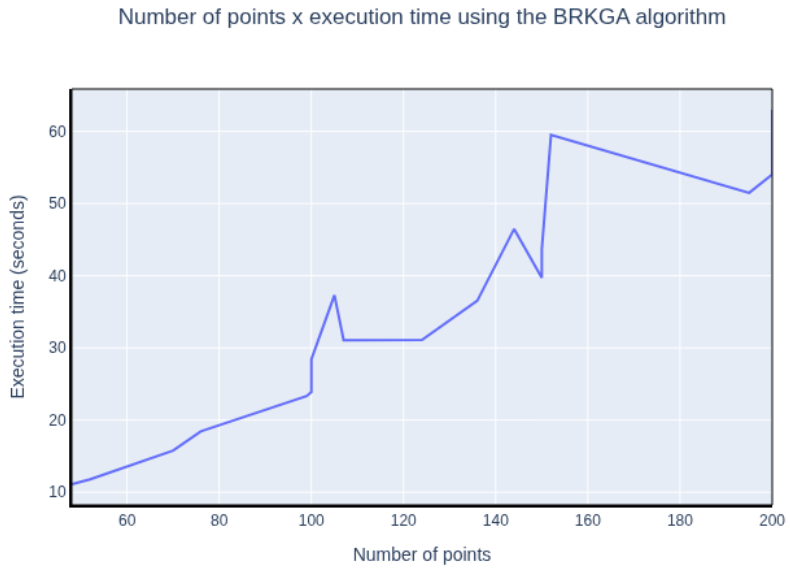

Figure 4. Input Size X Execution time in second test

3.3. Third test

- Number of Generations: 100000
- Population size: 200
- Elite Fraction: 25%
- Mutant Fraction: 10%
- Bias: 70%

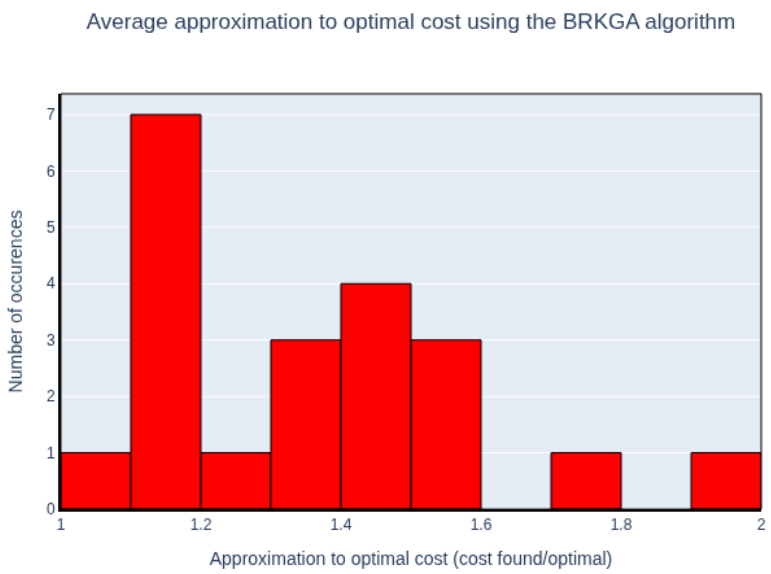

Figure 5. Histogram of approximation rates in third test

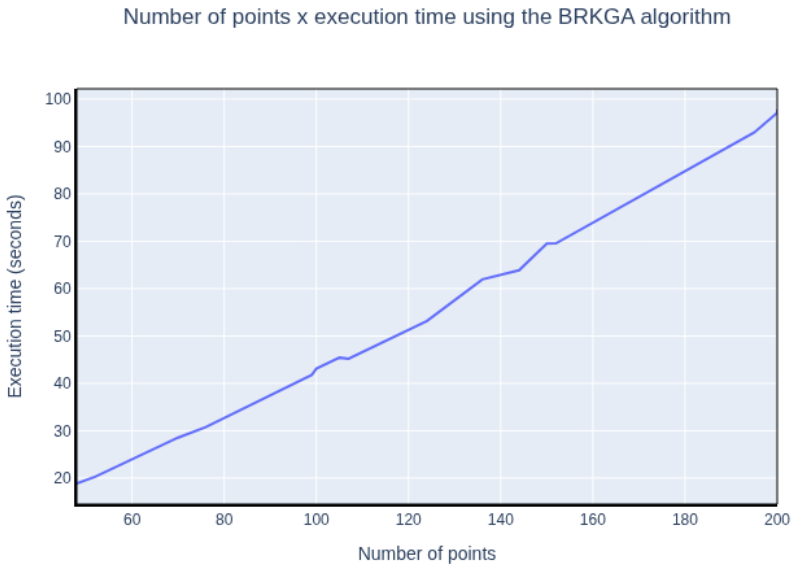

Figure 6. Input Size X Execution time in third test

As we can observe from the experiments, the BRKGA algorithm presents a strong dependency between its parameters and the quality of the obtained solution. In general, the greater the number of generations, the better the approximation to the optimal tour, since the algorithm has more opportunities to explore and refine the population. However, this improvement comes at the cost of an increased execution time.

Larger population sizes tend to preserve diversity and avoid premature convergence, often resulting in better solutions. Nevertheless, this also increases the computational cost of each generation, as more individuals need to be evaluated and sorted. The elite fraction determines how many of the best individuals are preserved from generation to generation. A higher elite fraction accelerates convergence, but may reduce diversity, potentially trapping the algorithm in local optima.

The mutant fraction, on the other hand, introduces randomness into the population, which helps to maintain diversity and escape suboptimal regions of the search space. Too few mutants can cause stagnation, while too many can disrupt convergence. Finally, the bias parameter controls the probability that an offspring inherits genes from an elite parent rather than a non-elite one. Higher bias values favor exploitation, while lower values promote exploration.

3.4. Complexity Analysis

Let P be the population size, G the number of generations, and n the number of vertices in the TSP instance. Each individual represents a vector of n random keys, which must be decoded into a tour by sorting these keys and then evaluating the total tour length. The decoding and evaluation process for a single individual requires $O(n \log n)$ time for sorting and $O(n)$ for computing the distance, which results in $O(n \log n)$. Since the population contains P individuals, the total cost of evaluating one generation is $O(P \times n \log n)$. Running the algorithm for G generations results in a time complexity of $O(G \times P \times n \log n)$. Regarding space complexity, the algorithm needs to store the population, where each individual contains n genes (random keys). Therefore, the memory requirement is $O(P \times n)$.

This means that increasing the number of generations G or the population size P improves the quality of the final solution but also linearly increases computational time and memory usage, as we can see in figures (2), (4) and (6). In contrast, smaller populations and fewer generations reduce runtime at the cost of solution accuracy. Thus, proper tuning of the BRKGA parameters is crucial to balance efficiency and solution quality.

4. Conclusion

As we could see, the BRKGA algorithm also has practical applications to solve the ETSP. Although exact algorithms can find the optimal tour, they quickly become infeasible as the number of points increases. The BRKGA, on the other hand, provides a good balance between solution quality and computational efficiency.

Since BRKGA is a metaheuristic, it does not guarantee an optimal answer, but it usually produces solutions that are very close to it, especially when properly tuned. Its main advantage is flexibility: it can handle large instances and adapt to different problem variations without major modifications. However, this comes at the cost of higher imple-

mentation complexity and the need to adjust several parameters, such as population size, mutation rate, and bias.

Therefore, for small or medium sets of points, classical approximation algorithms and constructive heuristics such as TATT or Christofides might be more straightforward and predictable. But when dealing with large or complex instances, where exact and deterministic methods become computationally expensive, the BRKGA algorithm becomes an excellent choice, capable of finding high-quality approximate tours in reasonable time.

References

[Gonçalves and Resende 2010] Gonçalves, J. F. and Resende, M. G. C. (2010). Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, 17(5):487–525.

[Reinelt 1991] Reinelt, G. (1991). Tsplib—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384.